

Transformation Rules for LinkedBrainz RDF View

Table 1: Global URI predicates used

URI_artist(a,x) := hasURI("http://musicbrainz.org/artist/", [a.gid, "#_"], x)

URI_area(ar,x) := hasURI("http://musicbrainz.org/area/", [ar.gid, "#_"], x)

URI_label(l,x) := hasURI("http://musicbrainz.org/label/", [l.gid, "#_"], x)

URI_medium(m,x) := hasURI("http://musicbrainz.org/record/", [m.id, "#_"], x)

URI_place(p,x) := hasURI("http://musicbrainz.org/place/", [wpid, "#_"], x)

URI_recording(r,x) := hasURI("http://musicbrainz.org/recording/", [r.gid, "#_"], x)

URI_release(r,x) := hasURI("http://musicbrainz.org/release/", [r.gid, "#_"], x)

URI_signalGroup(rg,x) := hasURI("http://musicbrainz.org/signal-group/", [rg.gid, "#_"], x)

URI_track(t,x) := hasURI("http://musicbrainz.org/track/", [t.id, "#_"], x)

URI_tag(tg,x) := hasURI("http://musicbrainz.org/tag/", [tg.name, "#_"], x)

URI_work(w,x) := hasURI(<http://musicbrainz.org/work/>, [w.gid, "#_"], x)

URI_v_releaseEvent(re,x) := hasURI("http://musicbrainz.org/releaseEvent/", [re.release_gid, re.area_gid, "#_"], x).

URI_v_releaseTagging(t,x) := hasURI(<http://musicbrainz.org/release-tagging/>,
[t.entity_id, t.tag_id, "#_"], x)

URI_artistTagging(t,x) := hasURI(<http://musicbrainz.org/artist-tagging/>,
[t.entity_id, t.tag_id, "#_"], x)

```

URI_v_label_tagging(t,x) := hasURI( "http://musicbrainz.org/label-tagging/",
                                     [t.entity_id, t.tag_id, "#_"], x)

URI_v_place_tagging(t,x) := hasURI( "http://musicbrainz.org/place-tagging/",
                                     [t.entity_id, t.tag_id, "#_"], x)

URI_v_recording_tagging(t,x) := hasURI( "http://musicbrainz.org/record-tagging/",
                                          [t.entity_id, t.tag_id, "#_"], x)

URI_v_work_tagging(t,x) := hasURI( "http://musicbrainz.org/work-tagging/",
                                    [t.entity_id, t.tag_id, "#_"], x)

```

Transformation Rule URI and Named Graph URI

Each Transformation Rule Ψ is identified by a URI.

The URI of a Transformation Rule with identifier ψ_id is constructed by concatenating:

1. the fixed URI prefix: "<http://musicbrainz.org/graph/mapping/>"
2. the transformation rule identifier ψ_id .

Formally: $URI(\Psi) =$

"<http://musicbrainz.org/graph/mapping/>" || rule_id(Ψ)

The named graph URI associated with Ψ is the same URI:

$URI_G(\Psi) = URI(\Psi)$

Example:

For the transformation rule: `psi_artist_01` the rule URI is:

http://musicbrainz.org/graph/mapping/psi_artist_01

and the named graph containing the RDF contribution generated by this rule is also:

GRAPH <http://musicbrainz.org/graph/mapping/psi_artist_01>

Therefore, each transformation rule Ψ is associated with exactly one named graph, and both the rule and its RDF contribution are identified by the same URI.

This design simplifies provenance management and incremental maintenance because the asynchronous worker can directly retrieve all RDF quads generated by a transformation rule Ψ by querying its associated named graph:

```
SELECT ?s ?p ?o
```

```
WHERE {
```

```
    GRAPH <URI( $\Psi$ )> { ?s ?p ?o }}
```

Hence, the named graph serves simultaneously as:

the provenance container of all RDF quads generated by Ψ ; and

the physical representation of the RDF contribution associated with Ψ .

This allows maintenance operations to be performed at the granularity of individual transformation rules, which is particularly useful for computing rule-level changesets

TRs Grouped by Pivot Relations

Notation: fk^{-1} indicates inverse traversal of the actual schema foreign key.

This table groups the generated Transformation Rules by pivot relation. The formulas follow the revised TR patterns: CTR rules generate class assertions, DTR rules generate literal values, and OTR rules generate URI resources.

Pivot relation	Rule	Type	Formula
----------------	------	------	---------

artist

<code>`artist`</code>	<code>`ψ_artist_01`</code>	CTR	<code>`mo:MusicArtist(s) ← artist(a), URI_artist(a,s).`</code>
-----------------------	----------------------------	-----	--

<code>`ψ_artist_02`</code>	Local-DTR	<code>`mo:musicbrainz_guid(s,v) ← artist(a), URI_artist(a,s), nonNull(a.gid), RDFLiteral(a.gid,"gid","artist",v).`</code>
----------------------------	-----------	---

<code>`ψ_artist_03`</code>	Local-DTR	<code>`foaf:name(s,v) ← artist(a), URI_artist(a,s), nonNull(a.name), RDFLiteral(a.name,"name","artist",v).`</code>
----------------------------	-----------	--

```

| `` | `ψ_artist_04` | Local-DTR | `ov:sortLabel(s,v) ← artist(a), URI_artist(a,s),
nonNull(a.sort_name), RDFLiteral(a.sort_name,"sort_name","artist",v).` |

| `` | `ψ_artist_05` | CTR | `mo:SoloMusicArtist(s) ← artist(a), URI_artist(a,s), a.type = 1.` |

| `` | `ψ_artist_06` | CTR | `mo:MusicGroup(s) ← artist(a), URI_artist(a,s), a.type = 2.` |

| `` | `ψ_artist_07` | Path-DTR | `foaf:gender(s,v) ← artist(a), URI_artist(a,s), gender(g),
nonNull(g.name), lower(g.name,u), RDFLiteral(u,"name","gender",v), [artist_fk_gender](a,g).` |

| `` | `ψ_artist_08` | OTR | `foaf:based_near(s,o) ← artist(a), URI_artist(a,s), area(ar),
URI_area(ar,o), [artist_fk_area](a,ar).` |

| `` | `ψ_artist_09` | OTR | `foaf:made(s,o) ← artist(a), URI_artist(a,s), release_group(rg),
URI_signalGroup(rg,o), [artist_credit_name_fk_artist^-1, artist_credit_name_fk_artist_credit,
release_group_fk_artist_credit^-1](a,rg).` |

| `` | `ψ_artist_10` | OTR | `foaf:made(s,o) ← artist(a), URI_artist(a,s), release(r),
URI_release(r,o), [artist_credit_name_fk_artist^-1, artist_credit_name_fk_artist_credit,
release_fk_artist_credit^-1](a,r).` |

| `` | `ψ_artist_11` | OTR | `foaf:made(s,o) ← artist(a), URI_artist(a,s), track(t), URI_track(t,o),
[artist_credit_name_fk_artist^-1, artist_credit_name_fk_artist_credit,
track_fk_artist_credit^-1](a,t).` |

| `` | `ψ_artist_12` | OTR | `foaf:made(s,o) ← artist(a), URI_artist(a,s), recording(r),
URI_recording(r,o), [artist_credit_name_fk_artist^-1, artist_credit_name_fk_artist_credit,
recording_fk_artist_credit^-1](a,r).` |

| `` | `ψ_artist_13` | Path-DTR | `rdfs:comment(s,v) ← artist(a), URI_artist(a,s), annotation(an),
nonNull(an.text), RDFLiteral(an.text,"text","annotation",v), [artist_annotation_fk_artist^-1,
artist_annotation_fk_annotation](a,an).` |

| `` | `ψ_artist_14` | OTR-external | `owl:sameAs(s,o) ← artist(a), URI_artist(a,s), url(u),
similarTo(u.url,"http://(de|el|en|es|ko|pl|pt).wikipedia.org/wiki/%"),
replace(u.url,"wikipedia.org/wiki","dbpedia.org/resource",u1),
replace(u1,"http://en.", "http://",u2), DBpediaURI(u2,o), [l_artist_url_fk_entity0^-1,
l_artist_url_fk_entity1](a,u).` |

| `release_group` | `ψ_release_group_01` | OTR-external | `rdfs:seeAlso(s,o) ← release_group(rg),
URI_signalGroup(rg,s), url(u),
similarTo(u.url,"http://(de|el|en|es|ko|pl|pt).wikipedia.org/wiki/%"),
replace(u.url,"wikipedia.org/wiki","dbpedia.org/resource",u1),
replace(u1,"http://en.", "http://",u2), IRIValue(u2,o), [l_release_group_url_fk_entity0^-1,
l_release_group_url_fk_entity1](rg,u).` |

| `` | `ψ_release_group_02` | CTR | `mo:SignalGroup(s) ← release_group(rg),
URI_signalGroup(rg,s).` |

```

| `` | `ψ\_release\_group\_03` | Local-DTR | `mo:musicbrainz_guid(s,v) ← release_group(rg),
URI_signalGroup(rg,s), nonNull(rg.gid), RDFLiteral(rg.gid,"gid","release_group",v).` |

| `` | `ψ\_release\_group\_04` | Local-DTR | `dc:title(s,v) ← release_group(rg),
URI_signalGroup(rg,s), nonNull(rg.name), RDFLiteral(rg.name,"name","release_group",v).` |

| `` | `ψ\_release\_group\_05` | Path-DTR | `rdfs:comment(s,v) ← release_group(rg),
URI_signalGroup(rg,s), annotation(an), nonNull(an.text),
RDFLiteral(an.text,"text","annotation",v), [release_group_annotation_fk_release_group^-1,
release_group_annotation_fk_annotation](rg,an).` |

| **area** | `ψ\_area\_01` | OTR-external | `owl:sameAs(s,o) ← area(ar), URI_area(ar,s), url(u),
similarTo(u.url,"http://(de|el|en|es|ko|pl|pt).wikipedia.org/wiki/%"),
replace(u.url,"wikipedia.org/wiki","dbpedia.org/resource",u1),
replace(u1,"http://en.", "http://", u2), DBpediaURI(u2,o), [l_area_url_fk_entity0^-1,
l_area_url_fk_entity1](ar,u).` |

| **label** | `ψ\_label\_01` | CTR | `mo:Label(s) ← label(l), URI\_label(l,s).` |

| `` | `ψ\_label\_02` | Local-DTR | `mo:musicbrainz_guid(s,v) ← label(l), URI_label(l,s),
nonNull(l.gid), RDFLiteral(l.gid,"gid","label",v).` |

| `` | `ψ\_label\_03` | OTR | `foaf:based_near(s,o) ← label(l), URI_label(l,s), area(ar),
URI_area(ar,o), [label_fk_area](l,ar).` |

| `` | `ψ\_label\_04` | Path-DTR | `rdfs:comment(s,v) ← label(l), URI_label(l,s), annotation(an),
nonNull(an.text), RDFLiteral(an.text,"text","annotation",v), [label_annotation_fk_label^-1,
label_annotation_fk_annotation](l,an).` |

| **medium** | `ψ\_medium\_01` | CTR | `mo:Record(s) ← medium(m), URI\_medium(m,s).` |

| `` | `ψ\_medium\_02` | Local-DTR | `mo:track_count(s,v) ← medium(m), URI_medium(m,s),
nonNull(m.track_count), RDFLiteral(m.track_count,"track_count","medium",v).` |

| `` | `ψ\_medium\_03` | OTR | `mo:track(s,o) ← medium(m), URI_medium(m,s), track(t),
URI_track(t,o), [track_fk_medium^-1](m,t).` |

| `` | `ψ\_medium\_04` | Path-DTR | `dc:format(s,v) ← medium(m), URI_medium(m,s),
medium_format(f), nonNull(f.name), RDFLiteral(f.name,"name","medium_format",v),
[medium_fk_format](m,f).` | j

| **place** | `ψ\_place\_01` | CTR | `geo:SpatialThing(s) ← place(p), URI\_place(p,s).` |

| `` | `ψ\_place\_02` | Path-DTR | `rdfs:comment(s,v) ← place(p), URI_place(p,s), annotation(an),
nonNull(an.text), RDFLiteral(an.text,"text","annotation",v), [place_annotation_fk_place^-1,
place_annotation_fk_annotation](p,an).` |

| **recording** | `ψ\_recording\_01` | CTR | `mo:Signal(s) ← recording(r), URI\_recording(r,s).` |

| `` | `ψ\_recording\_02` | Local-DTR | `mo:musicbrainz_guid(s,v) ← recording(r),
URI_recording(r,s), nonNull(r.gid), RDFLiteral(r.gid,"gid","recording",v).` |

| `` | `ψ\_recording\_03` | Local-DTR | `dc:title(s,v) ← recording(r), URI_recording(r,s),
nonNull(r.name), RDFLiteral(r.name,"name","recording",v).` |

| `` | `ψ\_recording\_04` | Path-DTR | `rdfs:comment(s,v) ← recording(r), URI_recording(r,s),
annotation(an), nonNull(an.text), RDFLiteral(an.text,"text","annotation",v),
[recording_annotation_fk_recording^-1, recording_annotation_fk_annotation](r,an).` |

| **release** | `ψ\_release\_01` | CTR | `mo:Release(s) ← release(r), URI\_release(r,s).` |

| `` | `ψ\_release\_02` | Local-DTR | `mo:musicbrainz_guid(s,v) ← release(r), URI_release(r,s),
nonNull(r.gid), RDFLiteral(r.gid,"gid","release",v).` |

| `` | `ψ\_release\_03` | Local-DTR | `dc:title(s,v) ← release(r), URI_release(r,s), nonNull(r.name),
RDFLiteral(r.name,"name","release",v).` |

| `` | `ψ\_release\_04` | OTR | `mo:record(s,o) ← release(r), URI_release(r,s), medium(m),
URI_medium(m,o), [medium_fk_release^-1](r,m).` |

| `` | `ψ\_release\_05` | OTR | `mo:release_type / **dct:isPartOf(s,o)** ← release(r), URI_release(r,s),
release_group(rg), URI_signalGroup(rg,o), [release_fk_release_group](r,rg).` |

| `` | `ψ\_release\_06` | OTR | `mo:label(s,o) ← release(r), URI_release(r,s), label(l), URI_label(l,o),
[release_label_fk_release^-1, release_label_fk_label](r,l).` |

| `` | `ψ\_release\_07` | Path-DTR | `rdfs:comment(s,v) ← release(r), URI_release(r,s),
annotation(an), nonNull(an.text), RDFLiteral(an.text,"text","annotation",v),
[release_annotation_fk_release^-1, release_annotation_fk_annotation](r,an).` |

| **v_release_event** | `ψ\_v\_release\_event\_01` | Derived-Pivot CTR | `mo:ReleaseEvent(s) ←
v_release_event(re), URI_releaseEvent(re,s).` |

| `` | `ψ\_v\_release\_event\_02` | OTR | `mo:release(s,o) ← v_release_event(re),
URI_releaseEvent(re,s), release(r), URI_release(r,o), [v_release_event_fk_release](re,r).` |

| `` | `ψ\_v\_release\_event\_03` | Local-DTR | `dc:date(s,v) ← v_release_event(re),
URI_releaseEvent(re,s), nonNull(re.date_year), nonNull(re.date_month), nonNull(re.date_day),
DateLiteral(re.date_year,re.date_month,re.date_day,v).` |

| `` | `ψ\_v\_release\_event\_04` | Local-DTR | `dc:date(s,v) ← v_release_event(re),
URI_releaseEvent(re,s), nonNull(re.date_year), nonNull(re.date_month),
YearMonthLiteral(re.date_year,re.date_month,v).` |

| `` | `ψ\_v\_release\_event\_05` | Local-DTR | `dc:date(s,v) ← v_release_event(re),
URI_releaseEvent(re,s), nonNull(re.date_year), YearLiteral(re.date_year,v).` |

| `` | `ψ\_v\_release\_event\_06` | OTR | `event:place(s,o) ← v_release_event(re),
URI_releaseEvent(re,s), area(ar), URI_area(ar,o), [v_release_event_fk_area](re,ar).` |

| **tag** | `ψ\_tag\_01` | CTR | `muto:Tag(s) ← tag(tg), URI\_tag(tg,s).` |

| `` | `ψ\_tag\_02` | Local-DTR | `dc:title(s,v) ← tag(tg), URI_tag(tg,s), nonNull(tg.name),
RDFLiteral(tg.name,"name","tag",v).` |

| **v_artist_tagging** | `ψ\_v\_artist\_tagging\_01` | Derived-Pivot CTR | `muto:Tagging(s) ←
v_artist_tagging(x), URI_artist_tagging(x,s).` |

| `` | `ψ\_v\_artist\_tagging\_02` | OTR | `muto:taggedResource(s,o) ← v_artist_tagging(x),
URI_artist_tagging(x,s), artist(e), URI_artist(e,o), [v_artist_tagging_fk_artist](x,e).` |

| `` | `ψ\_v\_artist\_tagging\_03` | OTR | `muto:hasTag(s,o) ← v_artist_tagging(x),
URI_artist_tagging(x,s), tag(tg), URI_tag(tg,o), [v_artist_tagging_fk_tag](x,tg).` |

| **v_label_tagging** | `ψ\_v\_label\_tagging\_01` | Derived-Pivot CTR | `muto:Tagging(s) ←
v_label_tagging(x), URI_label_tagging(x,s).` |

| `` | `ψ\_v\_label\_tagging\_02` | OTR | `muto:taggedResource(s,o) ← v_label_tagging(x),
URI_label_tagging(x,s), label(e), URI_label(e,o), [v_label_tagging_fk_label](x,e).` |

| `` | `ψ\_v\_label\_tagging\_03` | OTR | `muto:hasTag(s,o) ← v_label_tagging(x),
URI_label_tagging(x,s), tag(tg), URI_tag(tg,o), [v_label_tagging_fk_tag](x,tg).` |

| **v_place_tagging** | `ψ\_v\_place\_tagging\_01` | Derived-Pivot CTR | `muto:Tagging(s) ←
v_place_tagging(x), URI_place_tagging(x,s).` |

| `` | `ψ\_v\_place\_tagging\_02` | OTR | `muto:taggedResource(s,o) ← v_place_tagging(x),
URI_place_tagging(x,s), place(e), URI_place(e,o), [v_place_tagging_fk_place](x,e).` |

| `` | `ψ\_v\_place\_tagging\_03` | OTR | `muto:hasTag(s,o) ← v_place_tagging(x),
URI_place_tagging(x,s), tag(tg), URI_tag(tg,o), [v_place_tagging_fk_tag](x,tg).` |

| **v_recording_tagging** | `ψ\_v\_recording\_tagging\_01` | Derived-Pivot CTR | `muto:Tagging(s) ←
v_recording_tagging(x), URI_recording_tagging(x,s).` |

| `` | `ψ\_v\_recording\_tagging\_02` | OTR | `muto:taggedResource(s,o) ← v_recording_tagging(x),
URI_recording_tagging(x,s), recording(e), URI_recording(e,o),
[v_recording_tagging_fk_recording](x,e).` |

| `` | `ψ\_v\_recording\_tagging\_03` | OTR | `muto:hasTag(s,o) ← v_recording_tagging(x),
URI_recording_tagging(x,s), tag(tg), URI_tag(tg,o), [v_recording_tagging_fk_tag](x,tg).` |

| **v_release_tagging** | `ψ\_v\_release\_tagging\_01` | Derived-Pivot CTR | `muto:Tagging(s) ←
v_release_tagging(x), URI_release_tagging(x,s).` |

| `` | `ψ\_v\_release\_tagging\_02` | OTR | `muto:taggedResource(s,o) ← v_release_tagging(x),
URI_release_tagging(x,s), release(e), URI_release(e,o), [v_release_tagging_fk_release](x,e).` |

| `` | `ψ\_v\_release\_tagging\_03` | OTR | `muto:hasTag(s,o) ← v_release_tagging(x),
URI_release_tagging(x,s), tag(tg), URI_tag(tg,o), [v_release_tagging_fk_tag](x,tg).` |

| `v\_work\_tagging` | `ψ\_v\_work\_tagging\_01` | Derived-Pivot CTR | `muto:Tagging(s) ←
v_work_tagging(x), URI_work_tagging(x,s).` |

| `` | `ψ\_v\_work\_tagging\_02` | OTR | `muto:taggedResource(s,o) ← v_work_tagging(x),
URI_work_tagging(x,s), work(e), URI_work(e,o), [v_work_tagging_fk_work](x,e).` |

| `` | `ψ\_v\_work\_tagging\_03` | OTR | `muto:hasTag(s,o) ← v_work_tagging(x),
URI_work_tagging(x,s), tag(tg), URI_tag(tg,o), [v_work_tagging_fk_tag](x,tg).` |

| `track` | `ψ\_track\_01` | CTR | `mo:Track(s) ← track(t), URI\_track(t,s).` |

| `` | `ψ\_track\_02` | Local-DTR | `mo:track_number(s,v) ← track(t), URI_track(t,s),
nonNull(t.position), RDFLiteral(t.position,"position","track",v).` |

| `` | `ψ\_track\_03` | Local-DTR | `dc:title(s,v) ← track(t), URI_track(t,s), nonNull(t.name),
RDFLiteral(t.name,"name","track",v).` |

| `` | `ψ\_track\_04` | Local-DTR | `dc:title(s,v) ← track(t), URI_track(t,s), nonNull(t.name),
RDFLiteral(t.name,"name","track",v).` |

| `` | `ψ\_track\_05` | Local-DTR | `mo:duration(s,v) ← track(t), URI_track(t,s), nonNull(t.length),
RDFLiteral(t.length,"length","track",v).` |

| `` | `ψ\_track\_06` | OTR | `mo:publication_of(s,o) ← track(t), URI_track(t,s), recording(r),
URI_recording(r,o), [track_fk_recording](t,r).` |

| `work` | `ψ\_work\_01` | CTR | `mo:MusicalWork(s) ← work(w), URI\_work(w,s).` |

| `` | `ψ\_work\_02` | Local-DTR | `mo:musicbrainz_guid(s,v) ← work(w), URI_work(w,s),
nonNull(w.gid), RDFLiteral(w.gid,"gid","work",v).` |

| `` | `ψ\_work\_03` | Path-DTR | `rdfs:comment(s,v) ← work(w), URI_work(w,s), annotation(an),
nonNull(an.text), RDFLiteral(an.text,"text","annotation",v), [work_annotation_fk_work^-1,
work_annotation_fk_annotation](w,an).` |

Normalized derived pivots introduced for non-object-preserving maps

View [v_release_event](#)

```
CREATE VIEW v_release_event AS
SELECT
rc.release AS release_id,
r.gid AS release_gid,
rc.country AS area_id,
ar.gid AS area_gid,
rc.date_year,
rc.date_month,
rc.date_day
FROM release_country rc
JOIN release r
ON rc.release = r.id
JOIN area ar
ON rc.country = ar.id;
```

– v_release_event(release_id, release_gid, country_area_id, country_gid,
date_year, date_month, date_day), derived from release , release_country,
country_area , area, key (release_id, country_area_id).

=====

Views:

v_artist_tagging, *v_label_tagging*, *v_place_tagging*, *v_recording_tagging*,
v_release_tagging, *v_work_tagging*,

each derived from the corresponding *_tag association table joined with the entity table and tag, key (entity_id, tag_id).

CREATE VIEW v_artist_tagging AS

```
SELECT
    at.artist AS entity_id,
    a.gid    AS entity_gid,
    at.tag   AS tag_id,
    tg.name  AS tag_name,
    at.count,
    at.last_updated
FROM artist_tag at
JOIN artist a
    ON at.artist = a.id
JOIN tag tg
    ON at.tag = tg.id;
```

CREATE VIEW v_label_tagging AS

```
SELECT
    lt.label AS entity_id,
    l.gid    AS entity_gid,
    lt.tag   AS tag_id,
    tg.name  AS tag_name,
    lt.count,
    lt.last_updated
FROM label_tag lt
JOIN label l
    ON lt.label = l.id
JOIN tag tg
    ON lt.tag = tg.id;
```

```
CREATE VIEW v_place_tagging AS  
SELECT
```

```
    pt.place AS entity_id,  
    p.gid AS entity_gid,  
    pt.tag AS tag_id,  
    tg.name AS tag_name,  
    pt.count,  
    pt.last_updated  
FROM place_tag pt  
JOIN place p  
    ON pt.place = p.id  
JOIN tag tg  
    ON pt.tag = tg.id;
```

```
CREATE VIEW v_recording_tagging AS  
SELECT
```

```
    rt.recording AS entity_id,  
    r.gid AS entity_gid,  
    rt.tag AS tag_id,  
    tg.name AS tag_name,  
    rt.count,  
    rt.last_updated  
FROM recording_tag rt  
JOIN recording r  
    ON rt.recording = r.id  
JOIN tag tg  
    ON rt.tag = tg.id;
```

```
CREATE VIEW v_work_tagging AS  
SELECT
```

```
    wt.work AS entity_id,  
    w.gid AS entity_gid,  
    wt.tag AS tag_id,  
    tg.name AS tag_name,  
    wt.count,  
    wt.last_updated  
FROM work_tag wt  
JOIN work w  
    ON wt.work = w.id  
JOIN tag tg  
    ON wt.tag = tg.id;
```